

Optimization of Pearson and Blur

DV1674 | Assignment 2



Alexander Järnström
aljr21@student.bth.se

Jesper Lennehag
jele22@student.bth.se

CONTENTS

I.	Introduction	1
II.	Method	1
III.	Pearson	1
III.I.	Sequential	1
III.II.	Parallelism	3
III.III.	What more?	4
IV.	Blur	4
V.	Discussion	4

I. INTRODUCTION

We got two programs, pearson and blur, both running different core algorithms with different performance issues. The purpose of this work is to optimize them improving their system utilization and time frame, performing tests to outline a baseline for comparison, and document the changes and their results.

II. METHOD

Each program was tested to determine a baseline using several profiling tools like:

- Valgrind: Base tool
 - Callgrind: Used for cache simulation and viewing instruction trees.
 - Massif: Used for memory utilization and analysis.
 - Memcheck: Used to check for leaks.
- Perf stat: used to gather system information during runs. Used with `-d` for cache information and `--repeat 10` to run the test ten times and give an average.
- VTune: sanity checks.

After a baseline was determined different optimisation steps were taken, after which performance tests were run. The improvement would be removed if it worsened the performance.

Parallelism will be implemented into the program when the sequential version is optimal.

III. PEARSON

A simple bash script was written, `tests.sh`, with the purpose of making the testing and verification simpler. It has two modes: verify the output of the binary to previously established results and run the performance tests, which was used to extract the data. It ran all the tests on all the provided data files containing the various sizes of vectors, and later all threads. The parallel version of the binary is verified by the provided verification script, named `verify.sh`, making the verification part of `tests.sh` redundant and removed, only leaving the different performance tests.

III.I. SEQUENTIAL

The initial step taken was to enable compiler optimizations with the flags `-O2` and `-O3` where the one with best time was kept, `-O3` proved to be 527ms quicker than `-O2` and brought the total time down with 71.8% from the original, shown in Table 1. All the other stages are using the `-O3` flag when compiling.

Table 1: Changes between the Base and the O3 step for size 1024.

	Difference	Actual
Time (ms)	-16,170	4,320
Cache Miss (%)	7.41	7.89
CPU (%)	0.00	100.00
Memory (MB)	0.00	22.12
Instructions	-197542221927	31984950024

Next was the implementation of a cache, keeping all the results from the mean and magnitude methods in the Vector class. This was achieved by creating two arrays `mean_cache` and `magnitude_cache` which were initialized with the maximum value a double can take. It would check the cache, if the value equals the maximum a value for mean and magnitude would be calculated and then stored at the provided index, otherwise the value would be collected and used. This subtle change pulled the time down with another 32.67% from the previous step, Table 2, and 74.4% from the original.

Table 2: Changes between O3 and the cache step for size 1024.

	Difference	Actual
Time (ms)	-1,833	2,487
Cache Miss (%)	3.52	11.41
CPU (%)	-0.10	99.90
Memory (MB)	0.02	22.14
Instructions	-8746790499	23238159525

The third step took inspiration from locality, moving the calculations into the vector. VTune showed massive allocation amounts, for size 1024 the total reached 30.2 GB, where the biggest corporate is the constant copying and construction of new vectors when performing all the calculations. This step took the cache and *moved* it into the vector, or with other words replaced it with the vector itself. By realizing the original vector wasn't needed, all the calculations were changed to be preformed directly on the original. This replaced the original values by the calculated ones storing them in the vector, saving them for the dot method. This step lowered the allocation amount down to 103.9 MB for 1024, meaning the requests for more memory to the OS fell giving a positive difference of 40.31% from the caching step, Table 3, and 88.64% difference from the original.

Table 3: Changes between cache and the locality step for size 1024.

	Difference	Actual
Time (ms)	-1,478	1,008
Cache Miss (%)	-7.80	3.61
CPU (%)	-0.10	99.80
Memory (MB)	-0.02	22.12
Instructions	-17290556276	5947603249

We continued to loop unfolding, allowing the loops in the vector to do four calculations per iteration and eliminating dependencies by doing said calculations to temporary variables and then calculating the final result from them minimizing the pipeline stalls. This simple change lowered the time by 17.62% from the previous step and 90.2% from the original, leaving it at 704ms for size 1024, shown in Table 4.

Table 4: Changes between locality and the loop unfolding step for size 1024.

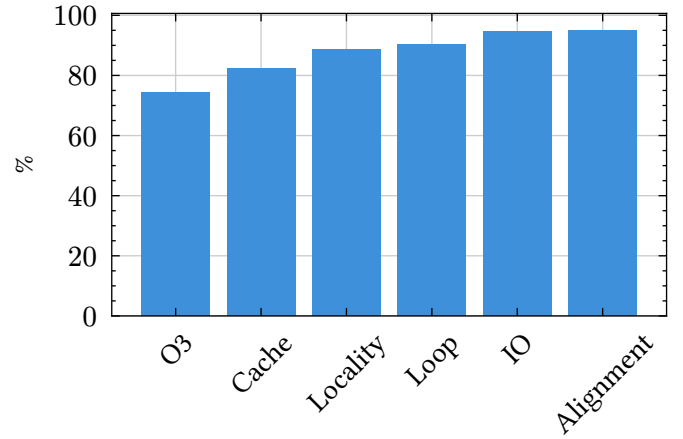
	Difference	Actual
Time (ms)	-304	704
Cache Miss (%)	0.00	3.61
CPU (%)	0.00	99.80
Memory (MB)	0.00	22.12
Instructions	-806078158	5141525091

The fifth step focused on IO the functions taking the most time now were dot, read, and write. write is heavily IO bound due to reading and writing file content to the disk, write was optimized by removing `std::endl` from the stream and replacing it with `\n` this allowed the stream to continue without clearing the buffer every iteration through the results. This gave us another 45.16% off the previous, Table 5, and a total of 94.64% from the original, giving us 404ms for 1024.

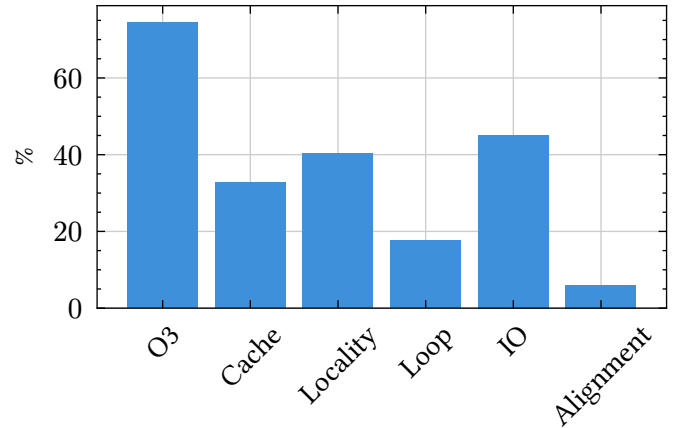
Table 5: Changes between loop unfolding and the IO step for size 1024.

	Difference	Actual
Time (ms)	-301	404
Cache Miss (%)	1.22	4.83
CPU (%)	-0.10	99.70
Memory (MB)	0.00	22.12
Instructions	-73611048	5067914043

The final step taken is vector alignment, by using `alignas(double)`. Aligning the vector class to the primary value in the data array, gave us a slight performance increase. This lowered the L1 cache misses by 0.21% and gave us a speed increase of 6.05% from the



Graph 1: Illustration of the total removed time in percent from the base.



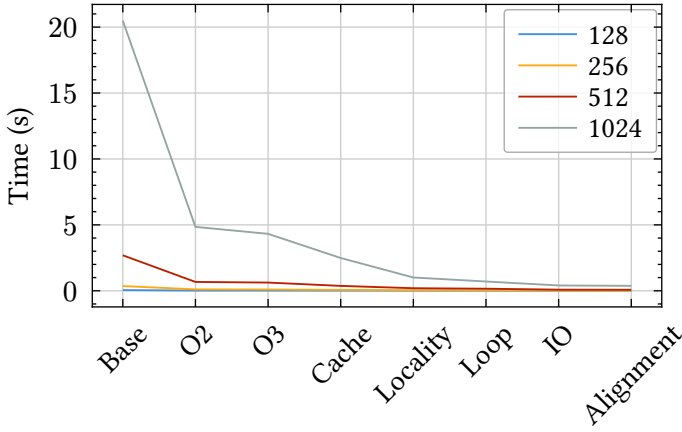
Graph 2: Illustration of the total removed time in percent from the previous step.

previous step, Table 6, and 94.93% from the original. Leaving us at the final time of 376ms for size 1024 compared to 20,490ms for the original with the same size.

Table 6: Changes between IO and the alignment step for size 1024.

	Difference	Actual
Time (ms)	-0.0278	376
Cache Miss (%)	-0.06	4.77
CPU (%)	0.00	99.70
Memory (MB)	0.00	22.12
Instructions	-7696245	5060217798

All the changes are illustrated in Graph 3 showing all the time differences through out the steps on different vector sizes. Due to the size 1024 over shadowing the other values, Graph 1 might be better to look at. The second graph shows the removed time in percent compared to the original while Graph 2 shows the removed time in percent from the previous step, or with other words, the impact.



Graph 3: Illustration of all the different sizes through all the taken steps.

III.II. PARALLELISM

There are some steps needed to achieve parallelism for the pearson program. Firstly a way for the user to specify the amount of wanted threads, secondly split up the work evenly, and lastly a safe way to get the results. To get the thread count a third argument for the binary was specified, thread count, which is used to separate the data evenly and create the wanted threads.

Separating the workload proved to be more of a hassle than expected, the dot method isn't run between all possible column pairs in the matrix, instead every column pairs with every column in front it. This can be visualized as *pair triangle*, meaning we can calculate the calculation count using:

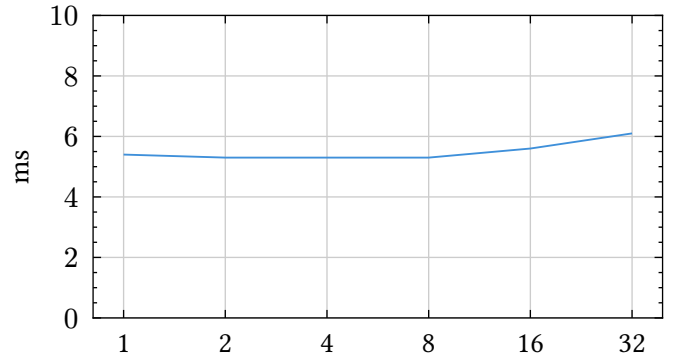
$$\text{pair count} = \frac{\text{vector size} * (\text{vector size} - 1)}{2}$$

, due to the last vector not being multiplied to it self, one is removed from it. The result of the multiplication will always be a multiple of two, thanks to the vector size. We then calculate the pair segment size which will be used to separate them between the threads:

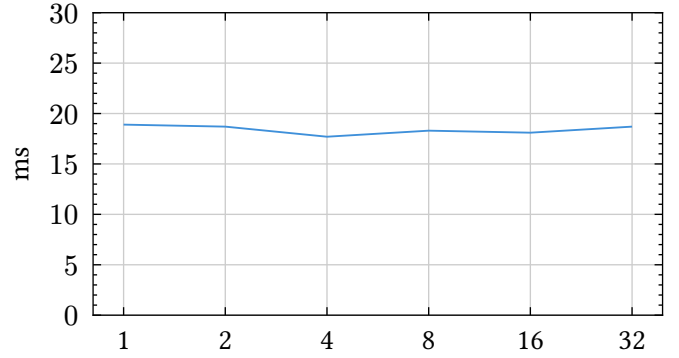
$$\text{segment size} = \frac{\text{pair count}}{\text{thread count}}$$

This is we needed to execute the dot method evenly across all the threads. But this excludes the prepeare method, leaving potential for parallelism. We continued to separate the dataset between the threads which proved to be simpler than the previous one, the only issue was the dependency between dot and prepeare. dot need prepeare to run on both the vectors in the calculation before executing. Giving us two choices: (1) running prepeare every time dot needed it, skipping the calculation if it was done, or (2) run prepeare on all the vectors before continuing. Due to all the needed mutex locks for the first option to mitigate all the race conditions, option two was chosen.

In conclusion, the program first separates the dataset between the threads, then it starts the threads which



Graph 4: Illustration of time over different thread counts for size 128.



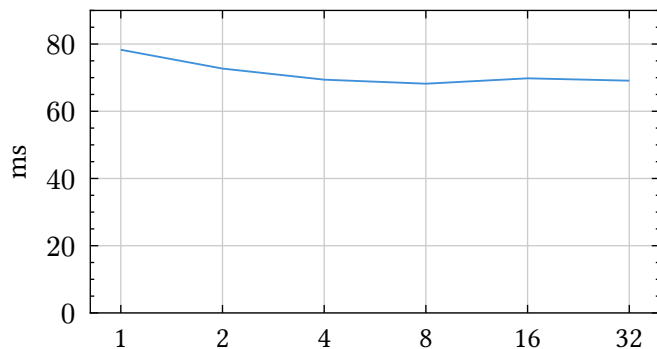
Graph 5: Illustration of time over different thread counts for size 256.

run the prepeare method on each vector. The main thread continues to add the vectors to pairs giving them their index for the result and then stops at a `pthread_barrier`, when all the other threads are done processing they'll reach the same barrier, which will let them proceed when all threads reach it. The threads continue to process the dot method and places the result in the result vector at the given index, they'll join the main thread when the processing is done.

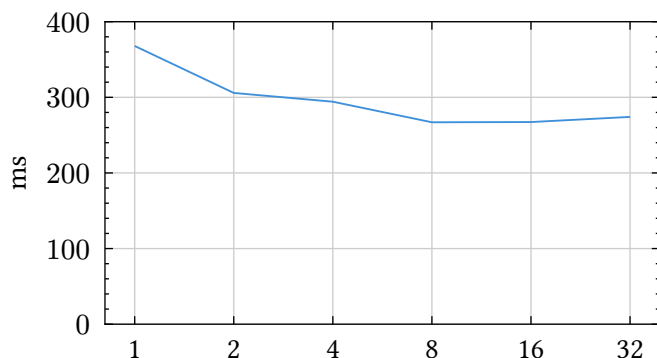
The same tests were run at the same sizes for each thread count, 1 to 32, giving the results shown through Graph 4 to Graph 7.

- Size 128 didn't showed any improvement through out the threads, with a best time of 5.3ms using two, four, and eight threads. This isn't better than the sequential version, meaning a this size threads aren't worth it.
- Size 256 showed slight improvement, with the greatest different from the sequential version at four threads at 17.7ms, which is a reduction of 1.1ms
- Size 512 shows a slightly better situation than the other two, with a best time of 68.2ms at eight threads, which is a reduction of 9.1ms.
- Size 1024 gave the best improvement, with a best time of 267ms at eight threads, which is a reduction of 109ms.

Though there simply aren't enough data to fully utilize the whole CPU, the greatest utilization reached was



Graph 6: Illustration of time over different thread counts for size 512.



Graph 7: Illustration of time over different thread counts for size 1024.

162.1% with 16 threads and size 1024, which didn't give the best results either way.

III.III. WHAT MORE?

Vectorisation using libraries such as `immintrin.h` could improve CPU utilization, performing several calculations simultaneously. This could give performance improvements, though the data sizes in this project seem to be too small to gain enough time to cover for the preparations. Moving the thread preparations to the read function could improve some overhead, the function itself could probably be improved with some other file reading or double conversion utility, though both read and write are heavily IO bound.

IV. BLUR

V. DISCUSSION
